Aim: Image processing using open CV.

Objective: To understand basic image processing operations using **OpenCV**

Software Requirements

- MATLAB / Python
- Libraries:
  - Open CV
  - NumPy
  - Matplotlib

Theory

Image Processing: It is the technique of applying operations on an image to improve its quality or extract meaningful information. It involves following operations on image.

1. Image Acquisition

It is the process of obtaining an image from a source such as a camera or file for further processing.

2. Grayscale Conversion

It converts a color image (RGB) into a single intensity image, reducing complexity and computation.

3. Image Smoothing (Gaussian Blur)

It reduces noise and small variations in the image using a Gaussian filter, making it smoother.

4. Edge Detection (Canny)

It identifies sharp changes in intensity, helping to detect edges and object boundaries.

5. Thresholding (Segmentation)

It converts a grayscale image into a binary image by separating pixels based on a threshold value.

6. Contour Detection

It detects and outlines the boundaries of objects present in the image.

Comparison between Open CV and MATLAB Functions

The following table shows the equivalence between functions used in **Open CV** and **MATLAB** for performing similar image processing operations.

Although both platforms perform the same tasks, their syntax and function names differ. Open CV (Python-based) provides more flexibility and is widely used in real-time and industry applications, whereas MATLAB offers built-in functions that are easier to use and more suitable for academic learning and visualization.

Open CV	**MATLAB**
cv2.imread()	imread()
cv2.cvtColor()	rgb2gray()
cv2.GaussianBlur()	imgaussfilt()
cv2.Canny()	edge(...,'Canny')
cv2.threshold()	imbinarize()
cv2.findContours()	boundaries()

Algorithm

1. Start
2. Upload the input image
3. Read the image using Open CV
4. Convert the image to grayscale
5. Apply Gaussian blur for smoothing
6. Perform Canny edge detection
7. Apply thresholding for segmentation
8. Detect contours in the segmented image
9. Draw contours on the original image
10. Display all results
11. Stop

Conclusion